

Donkey Car Anomalies Explained

Source: anomalies_guide.pdf, checked against the local implementation in manage.py, donkeycar/donkeycar/parts/dgym.py, donkeycar/donkeycar/parts/models/noise_generator.py, donkey-unity-sim/sdsim/Assets/Scripts/Car.cs, WheelPhys.cs, CameraSensor.cs, and TcpCarHandler.cs.

How anomaly injection works

All anomaly settings enter through repeated manage.py drive --meta key:value arguments. manage.py parses those key-value pairs into scalar settings such as friction_scale, mass_scale, blur_kernel, steering_gain, and steering_bias. The noise and name keys also label the output folder, so the run artifacts can be tied back to the anomaly configuration.

There are three implementation paths.

- Unity-side dynamics and camera-pose anomalies are sent once at simulator startup in a physics_config message. This path handles friction_scale, mass_scale, drag_force, and cam_pitch.
- Image-space sensing anomalies are applied inside DonkeyGymEnv.run_threaded before the image is returned to the controller. These require the noise label to contain the matching name, such as noise:mud, noise:blur, or noise:brightness.
- Control anomalies alter commands after the neural pilot has produced steering and throttle. steering_gain and steering_bias are applied in DriveMode, while cmd_latency delays the action stored in the simulator environment.

Dynamics anomalies

Friction Loss

Parameter: friction_scale. Guide range: 0.1 to 1.0. Typical settings are 0.9 for a painted lane, 0.6 for wet road, and 0.1 for icy road.

Relevant scripts to inspect: manage.py parses the --meta value and calls add_simulator; donkeycar/donkeycar/parts/dgym.py sends the physics_config message; donkey-unity-sim/sdsim/Assets/Scripts/tcp/TcpCarHandler.cs receives physics_config; donkey-unity-sim/sdsim/Assets/Scripts/Car.cs applies SetFrictionScale; donkey-unity-sim/sdsim/Assets/Scripts/WheelPhys.cs applies the scaled wheel friction each physics tick.

Friction loss reduces tire grip in Unity. When friction_scale is non-default, Python sends it in the physics_config message. TcpCarHandler receives that message and calls Car.SetFrictionScale. The car then writes the scale into each WheelPhys component.

WheelPhys applies the scale every physics tick. It takes each wheel collider's original forward and sideways friction stiffness, multiplies by the ground material's static friction, then multiplies by friction_scale. A lower value reduces both braking/drive traction and lateral cornering grip. In practice the car becomes easier to slide, less able to hold the lane during turns, and slower to recover after steering corrections.

Vehicle Mass

Parameter: `mass_scale`. Guide range: 1.0 to 2.0. Typical settings are 1.2, 1.5, and 2.0.

Relevant scripts to inspect: `manage.py` parses `mass_scale` and forwards it through `add_simulator`; `donkeycar/donkeycar/parts/dgym.py` includes `mass_scale` in `physics_config`; `donkey-unity-sim/sdsim/Assets/Scripts/tcp/TcpCarHandler.cs` dispatches the setting; `donkey-unity-sim/sdsim/Assets/Scripts/Car.cs` implements `SetMassScale`.

Vehicle mass changes the rigidbody mass in Unity. Python sends `mass_scale` in `physics_config`, and the Unity car sets `rb.mass` to `originalMass` multiplied by the scale.

Increasing mass does not directly change the pilot's requested steering or throttle. It changes the physics response to those commands. With more mass, the same motor torque produces less acceleration, braking takes longer, and steering corrections have more inertia to overcome. The controller may still output the same nominal commands, but the vehicle reacts more sluggishly and can overshoot because momentum carries it forward.

Asymmetric Drag

Parameter: `drag_force` in newtons. Guide range: 0 to 600. Typical settings are 100 for low tire pressure, 300 for crosswind, and 600 for tire blowout.

Relevant scripts to inspect: `manage.py` parses `drag_force` and `add_simulator` accepts it; `donkeycar/donkeycar/parts/dgym.py` includes `drag_force` in `physics_config`; `donkey-unity-sim/sdsim/Assets/Scripts/tcp/TcpCarHandler.cs` dispatches the setting; `donkey-unity-sim/sdsim/Assets/Scripts/Car.cs` implements `SetDragForce` and applies the force in `FixedUpdate`. Implementation note: in the checked `manage.py` drive setup, `drag_force` is parsed but the shown calls to `add_simulator` do not forward `drag_force`, so this is the first place to inspect if the guide command does not visibly activate drag.

Asymmetric drag adds a constant backward force at one off-center wheel in Unity. When `drag_force` reaches `DonkeyGymEnv`, Python sends it in `physics_config`. Unity stores the force in `Car.SetDragForce` and enables the asymmetric drag branch when the force is nonzero.

Every physics tick, the car checks its forward speed. If it is moving, it gets the world position of the front-right wheel collider and applies a force opposite the direction of travel at that point. Because the force is not applied at the center of mass, it creates a yaw torque in addition to slowing the car. That yaw bias pulls the car toward the affected side, which mimics a dragging tire, a strong side force, or a blowout-like failure. The guide notes that small forces are too weak for this 50 kg rigidbody, so visible behavior needs values in the hundreds of newtons.

Sensing anomalies

Mud Blob

Parameter: `occlusion_fraction`. Guide range: 0 to 1. Typical settings are 0.1, 0.4, and 0.9.

Donkey Car Anomalies Explained

Relevant scripts to inspect: `manage.py` parses `occlusion_fraction` and `noise`; `donkeycar/donkeycar/parts/dgym.py` checks whether "mud" is present in the noise label, chooses the fixed mud center, and calls the augmentor; `donkeycar/donkeycar/parts/models/noise_generator.py` implements `ImageAugmentor.add_mud_blob`. There is also a top-level `noise_generator.py` with the same augmentor logic.

Mud blob is an image-space anomaly, so it only activates when the noise label contains mud. In `run_threaded`, the code chooses a mud center the first time it sees the anomaly and then reuses that center for the rest of the run.

The blob area is derived from `occlusion_fraction`. The code computes `covered_pixels` as `image_width` times `image_height` times `occlusion_fraction`, converts that area into a circular radius, and draws an opaque brown circle. Pixels inside the circle are replaced by the mud color. Because the center is fixed after initialization, the occlusion behaves like something stuck to the camera lens rather than random dirt appearing independently on every frame.

Camera Pitch Shift

Parameter: `cam_pitch` in degrees. Guide range: -30 to 0. Typical settings are -5, -15, and -30.

Relevant scripts to inspect: `manage.py` parses `cam_pitch` and forwards it through `add_simulator`; `donkeycar/donkeycar/parts/dgym.py` sends it in `physics_config`; `donkey-unity-sim/sdsim/Assets/Scripts/tcp/TcpCarHandler.cs` dispatches the setting; `donkey-unity-sim/sdsim/Assets/Scripts/camera/CameraSensor.cs` implements `SetPitchOffset`.

Camera pitch is a Unity-side camera pose anomaly. Python sends `cam_pitch` in `physics_config`, `TcpCarHandler` receives it, and `CameraSensor.SetPitchOffset` adds the requested pitch offset to the camera sensor's original local rotation.

A negative pitch tilts the rendered camera view away from its normal calibration. The neural controller still receives an image with the same pixel dimensions, but the track geometry appears at a different vertical position and with a different perspective. This can make the learned lane-following features look shifted or compressed relative to training data.

Frame Dropping

Parameter: `frame_drop`, called `num_drop` internally. Guide range: 1 to 10. Typical settings are 2, 5, and 10.

Relevant scripts to inspect: `manage.py` parses `frame_drop` into `num_drop` and forwards it through `add_simulator`; `donkeycar/donkeycar/parts/dgym.py` stores `num_drop`, maintains `drop_counter` and `frozen_frame`, and returns the stale frame from `run_threaded`.

Frame dropping freezes the visual input for $N-1$ out of every N `run_threaded` calls. When the drop counter is zero, the current simulator frame is copied into `frozen_frame`. The counter then advances modulo N , and the frozen image is returned until the next refresh.

Donkey Car Anomalies Explained

Only the image is frozen. The simulator keeps stepping, telemetry keeps updating, and control commands keep being sent. This creates a stale-perception failure: the controller is steering from an older view of the track while the car has already moved to a new pose.

Visual Blur

Parameter: `blur_kernel`. Guide range: odd values from 3 to 15. Typical settings are 3, 7, and 15.

Relevant scripts to inspect: `manage.py` parses `blur_kernel` and `noise`; `donkeycar/donkeycar/parts/dgym.py` checks whether "blur" is present in the noise label and calls the augmentor; `donkeycar/donkeycar/parts/models/noise_generator.py` implements `ImageAugmentor.add_defocus` using OpenCV `GaussianBlur`. There is also a top-level `noise_generator.py` with the same augmentor logic.

Visual blur is an image-space anomaly, so the noise label must contain blur. `run_threaded` passes the camera image through `ImageAugmentor.add_defocus`, which applies OpenCV `GaussianBlur` with a square kernel of `blur_kernel` by `blur_kernel`.

The kernel must be odd because OpenCV's Gaussian blur requires odd kernel dimensions. Larger kernels spread each pixel over a wider neighborhood, removing high-frequency lane edges and texture cues. The controller still sees the same frame size and colors, but the crisp boundary evidence it learned from becomes weaker.

Brightness

Parameter: `brightness_coeff`. Guide range: 0.1 to 3.0. Typical settings are 0.5, 1.5, and 3.0.

Relevant scripts to inspect: `manage.py` parses `brightness_coeff` and `noise`; `donkeycar/donkeycar/parts/dgym.py` checks whether "brightness" is present in the noise label and calls the augmentor; `donkeycar/donkeycar/parts/models/noise_generator.py` implements `ImageAugmentor.change_brightness`. There is also a top-level `noise_generator.py` with the same augmentor logic.

Brightness is an image-space anomaly, so the noise label must contain brightness. The image is converted from RGB to HLS color space. The lightness channel is multiplied by `brightness_coeff`, clipped at 255, and converted back to RGB.

A coefficient below 1.0 darkens the scene, while a coefficient above 1.0 brightens it. Hue and saturation are preserved more directly than they would be with a raw RGB multiply, but saturation and contrast can still appear distorted after clipping. At high brightness, lane markings and road regions can wash together because many pixels saturate at maximum lightness.

Control anomalies

Command Latency

Donkey Car Anomalies Explained

Parameter: `cmd_latency` in frames. Guide range: N greater than or equal to 0. Typical settings are 2, 6, and 15.

Relevant scripts to inspect: `manage.py` parses `cmd_latency` and forwards it through `add_simulator`; `donkeycar/donkeycar/parts/dgym.py` creates the FIFO command queue and applies delayed actions in `run_threaded`.

Command latency delays the action applied to the simulator. In `run_threaded`, the current steering, throttle, and brake command is appended to a FIFO queue. Once the queue length exceeds `cmd_latency`, the oldest command is popped and becomes the active simulator action.

During the warm-up period before the queue is full, the simulator keeps its previous default action, effectively zeros at startup. After warm-up, the car is always executing commands from several frames ago. This is dangerous for lane following because steering corrections arrive after the visual situation that caused them has already passed.

Steering Misalignment

Parameter: `steering_bias`. Guide range: -0.5 to 0.5. Typical settings are 0.05, 0.15, and 0.4.

Relevant scripts to inspect: `manage.py` parses `steering_bias`, passes it into `DriveMode` as `steering_offset`, and defines the `DriveMode` class; `DriveMode.run` applies `steering_gain`, then `steering_offset`, then clamps the final steering value.

Steering misalignment is implemented as an additive offset on the final steering command. `manage.py` passes `steering_bias` into `DriveMode` as `steering_offset`. `DriveMode` first chooses user or pilot steering depending on mode, multiplies it by `steering_gain`, then adds `steering_offset` and clamps the result to the valid normalized steering range from -1 to 1.

A positive or negative bias means a nominally straight command no longer drives straight. The controller has to constantly compensate for the offset. If the bias is large, the command may also clip at the steering limits, reducing available correction in the opposite direction.

Power Steering Fail

Parameter: `steering_gain`. Guide range: 0 to 1. Typical settings are 0.8, 0.4, and 0.1.

Relevant scripts to inspect: `manage.py` parses `steering_gain`, passes it into `DriveMode`, and defines `DriveMode.run`; `DriveMode.run` multiplies the selected steering command by `steering_gain` before `steering_bias` is added.

Power steering failure reduces steering authority by multiplying the chosen steering command before bias is added. In `DriveMode`, the raw steering value is multiplied by `steering_gain`, then `steering_bias` is added, and the final value is clamped.

With `steering_gain` below 1.0, the same neural pilot output produces a smaller wheel angle. The car may not turn sharply enough for curves and may recover too slowly from lateral error. At very low gain, the controller can ask for full steering but still receive only a small fraction of the normal steering response.

GAN repair path

The guide also mentions cyclegan and pix2pix meta keys. These are not anomalies by themselves. They are optional image-repair models that can be loaded alongside an image-space sensing anomaly.

The intended pipeline is: simulator image, then image-space anomaly such as mud, blur, or brightness, then GAN repair, then the Keras controller. That ordering means the GAN sees the corrupted image and tries to translate it back toward a cleaner domain before the driving model makes a decision. This path is meaningful mainly for sensing anomalies, because dynamics and control anomalies do not corrupt the camera image.

Trial outputs

trials.py can repeat the same anomaly configuration multiple times. Each run writes a data folder labeled with environment, noise, and name. The key outputs are cte_values.csv, containing cross-track error per frame, and cte_plot.png, a plot of cross-track error over time. The simulator run stops after roughly sixty seconds or after sustained collision handling stops the loop.